02 — Procedural PHP

Useful basic functions

```php
echo "text";            // outputs text (several args allowed: echo $a, $b;)
print "text";           // same, returns 1 (rarely used)
var_dump($x);          // type + value, for debugging
print_r($array);       // readable dump of an array

isset($x);             // true if $x exists AND is not null
empty($x);             // true if $x doesn't exist OR is falsy ("", 0, "0", [], null, false)
unset($x);             // destroys the variable
```

`isset` vs `empty`, the classic trap:

`$x`	`isset($x)`	`empty($x)`
not defined	false	true
`null`	false	true
`""`	true	true
`"0"`	true	**true**
`0`	true	true
`"hello"`	true	false

Null coalescing operator, the modern shortcut:

```php
$name = $_GET['name'] ?? 'Guest';   // 'Guest' if not set or null
```

Conditions

```php
if ($age >= 18) {
    echo "Adult";
} elseif ($age >= 13) {
    echo "Teen";
} else {
    echo "Child";
}

$label = $age >= 18 ? 'Adult' : 'Minor';   // ternary
```

Logical operators: `&&`, `||`, `!`. (`and`, `or` exist but have a lower priority: avoid them.)

`switch` and `match`

```php
switch ($day) {
    case 'sat':
    case 'sun':
        echo 'Weekend';
        break;              // don't forget break!
    default:
        echo 'Weekday';
}

// PHP 8: match is an expression, uses ===, no break needed
$label = match ($day) {
    'sat', 'sun' => 'Weekend',
    default      => 'Weekday',
};
```

Loops

```php
for ($i = 1; $i <= 5; $i++) {
    echo $i;
}

$n = 10;
while ($n > 0) {
    $n -= 3;
}

do {
    $tries++;
} while ($tries < 3);     // runs at least once
```

`break` exits the loop, `continue` skips to the next iteration.

Arrays

PHP has **one** array type that is both a list and a dictionary (an ordered map).

```php
// Indexed array (list)
$fruits = ['apple', 'banana', 'cherry'];
echo $fruits[0];          // apple
$fruits[] = 'date';       // append
echo count($fruits);      // 4

// Associative array (dictionary)
$user = [
    'name'  => 'Alice',
    'email' => 'alice@example.com',
    'age'   => 32,
];
echo $user['name'];
$user['city'] = 'Tours';

// Nested
$users = [
    ['name' => 'Alice', 'age' => 32],
    ['name' => 'Bob',   'age' => 25],
];
echo $users[1]['name'];   // Bob
```

`foreach`

```php
foreach ($fruits as $fruit) {
    echo $fruit . PHP_EOL;
}

foreach ($user as $key => $value) {
    echo "$key: $value" . PHP_EOL;
}
```

Essential array functions

Function	Does
`count($a)`	number of elements
`in_array($v, $a)`	is the value present?
`array_key_exists($k, $a)`	is the key present?
`array_search($v, $a)`	key of a value (or `false`)
`array_push($a, $v)` / `array_pop($a)`	add / remove at the end
`array_shift($a)` / `array_unshift($a, $v)`	remove / add at the start
`array_merge($a, $b)`	combine arrays
`array_keys($a)` / `array_values($a)`	keys / values only
`array_slice($a, $offset, $length)`	part of an array
`sort` / `rsort`	sort values (re-indexes)
`asort` / `arsort`	sort values, **keep keys**
`ksort` / `krsort`	sort by key
`usort($a, fn)`	sort with your own comparison
`array_map(fn, $a)`	transform each element
`array_filter($a, fn)`	keep elements matching a condition
`array_sum($a)`	sum of values
`implode(', ', $a)` / `explode(',', $s)`	array ↔ string

Sort functions modify the array **in place** and return `true`, not the sorted array.

Functions

```php
function greet(string $name, string $greeting = 'Hello'): string
{
    return "$greeting, $name!";
}

echo greet('Alice');                  // Hello, Alice!
echo greet('Bob', 'Hi');              // Hi, Bob!
echo greet(greeting: 'Hey', name: 'Carol'); // named arguments (PHP 8)
```

- Types are optional but recommended: `int`, `float`, `string`, `bool`, `array`, `?string` (nullable), `int|float` (union), `void`, `mixed`.
- Variables inside a function are **local**: a function can't see variables from outside unless you pass them as parameters.
- Arguments are passed **by value** (copy). Use `&` to pass by reference: `function addOne(int &$n) { $n++; }`

Anonymous functions and arrow functions:

```php
$double = fn($x) => $x * 2;                    // arrow function, sees outer variables
$prices = array_map($double, [10, 20]);        // [20, 40]

$min = 15;
$expensive = array_filter($prices, fn($p) => $p > $min);  // [1 => 40] (keys are kept)
```

Strings and dates: the most used functions

```php
strlen('hello');                // 5 (bytes!)
mb_strlen('été');               // 3 (characters: use mb_ for accents)
strtoupper('abc'); strtolower('ABC'); ucfirst('alice');
trim('  hi  ');                 // "hi"
str_contains('hello world', 'world');   // true (PHP 8)
str_starts_with($s, 'http'); str_ends_with($s, '.jpg');
str_replace('cat', 'dog', 'my cat');    // "my dog"
substr('abcdef', 1, 3);         // "bcd"
strpos('abcdef', 'c');          // 2 (or false: compare with === false!)
sprintf('%s is %d years old', 'Alice', 32);
sprintf('%.2f', 3.14159);       // "3.14"

date('Y-m-d H:i');              // 2026-10-16 14:30
date('d/m/Y', strtotime('+1 week'));

$d = new DateTime('2026-12-25');
$today = new DateTime('today');
echo $today->diff($d)->days;    // days until Christmas
echo $d->format('l d F Y');     // Friday 25 December 2026
```

Now do the exercises.

02 — Exercises

All exercises are command-line scripts: `php file.php`.

2.1 — FizzBuzz, PHP edition

Print the numbers from 1 to 30. For multiples of 3 print `Fizz`, for multiples of 5 print `Buzz`, for multiples of both print `FizzBuzz`.

Write it once with `if/elseif`, then a second version using `match(true)`.

2.2 — Grades

Start from this array:

```php
$grades = [
    'Alice' => 15,
    'Bob'   => 9,
    'Chloe' => 18,
    'David' => 12,
    'Emma'  => 7,
];
```

1. Display each student with their grade: `Alice: 15/20`.
2. Compute and display the class average (2 decimals).
3. Display the best student and their grade.
4. Display the list of students who passed (grade >= 10), separated by commas. Use `array_filter` and `implode`.
5. Display the students sorted from best to worst grade, keeping names attached to grades.

2.3 — Functions

Write these functions with full type declarations, then call them to test:

1. `isPalindrome(string $word): bool` — `"Kayak"` → `true`. Ignore case.
2. `initials(string $fullName): string` — `"jean-paul sartre"` → `"J.S."`. Hint: `explode`, `strtoupper`, `substr`. (Bonus: `"J.P.S."` by also splitting on `-`.)
3. `applyDiscount(float $price, float $percent = 10): float` — returns the discounted price, rounded to 2 decimals. Call it once with named arguments.

2.4 — Shopping cart

```php
$cart = [
    ['product' => 'Keyboard', 'price' => 49.90, 'qty' => 1],
    ['product' => 'Mouse',    'price' => 19.50, 'qty' => 2],
    ['product' => 'Screen',   'price' => 189.00, 'qty' => 1],
];
```

1. Display one line per product: name, quantity, line total.
2. Display the cart total.
3. Using `array_map`, build an array containing only the product names.
4. Write a function `cartTotal(array $cart): float` and use it.

2.5 — Dates

1. Display today's date as `Friday 16 October 2026`.
2. Display the number of days until next 1st of January.
3. Given `$birthday = '1990-05-14'`, display the age in years (use `DateTime` and `diff`).